```
"""
BirdTrack Rare Species Deduplication Pipeline
==============================================
Processes BirdTrack-style exports to produce minimum/maximum individual
counts for rare species, applying behaviour-appropriate spatial and
temporal windows.

Usage:
    python dedup.py --input records.csv --species species_config.csv --output res

Column name mappings can be adjusted in config.py if your export differs.
"""

import pandas as pd
import numpy as np
from sklearn.cluster import DBSCAN
from math import radians
import argparse
import os
from datetime import timedelta

from config import COLUMN_MAP, BEHAVIOUR_WINDOWS
from species_config import load_species_config


# ---------------------------------------------------------------------------
# Utility functions
# ---------------------------------------------------------------------------

def haversine_distance_km(lat1, lon1, lat2, lon2):
    """Approximate distance in km between two points."""
    R = 6371.0
    dlat = radians(lat2 - lat1)
    dlon = radians(lon2 - lon1)
    a = (np.sin(dlat / 2) ** 2 +
         np.cos(radians(lat1)) * np.cos(radians(lat2)) * np.sin(dlon / 2) ** 2)
    return R * 2 * np.arcsin(np.sqrt(a))


def normalise_count(val):
    """Convert count field to integer; treat 'X' or blank as 1."""
    if pd.isna(val) or str(val).strip().upper() == 'X' or str(val).strip() == '':
        return 1
    try:
```

```
        return max(1, int(float(str(val).strip())))
    except ValueError:
        return 1


def parse_datetime(row, date_col, time_col):
    """Combine date and time columns into a single datetime."""
    date_str = str(row[date_col]).strip()
    time_str = str(row.get(time_col, '')).strip()
    if time_str in ('', 'nan', 'None'):
        time_str = '00:00'
    for fmt in ('%d/%m/%Y %H:%M', '%Y-%m-%d %H:%M', '%d/%m/%Y', '%Y-%m-%d'):
        try:
            return pd.to_datetime(f"{date_str} {time_str}", format=fmt)
        except ValueError:
            pass
    return pd.NaT


# ---------------------------------------------------------------------------
# Loading & normalising
# ---------------------------------------------------------------------------

def load_birdtrack(path, col_map):
    """Load a BirdTrack CSV export and rename columns to internal names."""
    df = pd.read_csv(path, encoding='utf-8-sig', low_memory=False)

    # Rename columns using map (only mapped columns kept)
    rename = {v: k for k, v in col_map.items() if v in df.columns}
    df = df.rename(columns=rename)

    required = ['species', 'date', 'site_name', 'observer_id', 'count']
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(
            f"After applying COLUMN_MAP, these required columns are still missing
            f"Available columns: {list(df.columns)}\n"
            f"Please update COLUMN_MAP in config.py."
        )

    df['count_int'] = df['count'].apply(normalise_count)

    # Parse datetime
    time_col = 'start_time' if 'start_time' in df.columns else 'date'
    df['datetime'] = df.apply(
        lambda r: parse_datetime(r, 'date', time_col), axis=1
    )
```

```
    df['date_only'] = df['datetime'].dt.date

    # Coordinates — may not always be present
    if 'latitude' in df.columns and 'longitude' in df.columns:
        df['latitude'] = pd.to_numeric(df['latitude'], errors='coerce')
        df['longitude'] = pd.to_numeric(df['longitude'], errors='coerce')
    else:
        df['latitude'] = np.nan
        df['longitude'] = np.nan

    return df


# ---------------------------------------------------------------------------
# Tier 1: Same visit / list deduplication
# ---------------------------------------------------------------------------

def dedupe_same_visit(df):
    """
    If a visit_id column exists, keep only the max count per
    visit + species combination (same observer, same outing).
    """
    if 'visit_id' not in df.columns:
        return df

    idx = (
        df.groupby(['visit_id', 'species'])['count_int']
        .idxmax()
    )
    return df.loc[idx].reset_index(drop=True)


# ---------------------------------------------------------------------------
# Tier 2 & 3: Spatiotemporal clustering
# ---------------------------------------------------------------------------

def build_time_clusters(group_df, time_window_hours):
    """
    Within a spatial cluster, assign time-based sub-clusters.
    Records within time_window_hours of each other are grouped together.
    Always returns a pd.Series aligned to group_df.index.
    """
    if len(group_df) == 1:
        return pd.Series([0], index=group_df.index)

    sorted_df = group_df.sort_values('datetime')
    labels = [0]
```

```
    current_label = 0
    prev_time = sorted_df.iloc[0]['datetime']

    for _, row in sorted_df.iloc[1:].iterrows():
        if pd.isna(row['datetime']) or pd.isna(prev_time):
            current_label += 1
        elif (row['datetime'] - prev_time) > timedelta(hours=time_window_hours):
            current_label += 1
        labels.append(current_label)
        prev_time = row['datetime']

    result = pd.Series(labels, index=sorted_df.index)
    return result.reindex(group_df.index)


def spatial_cluster(species_df, radius_km):
    """
    DBSCAN spatial clustering. Returns cluster labels.
    Records without coordinates are each assigned a unique cluster
    based on site_name only.
    """
    has_coords = (
        species_df['latitude'].notna() &
        species_df['longitude'].notna()
    )

    labels = np.full(len(species_df), -1, dtype=int)
    offset = 0

    if has_coords.sum() >= 1:
        coords = np.radians(
            species_df.loc[has_coords, ['latitude', 'longitude']].values
        )
        eps_rad = radius_km / 6371.0
        db = DBSCAN(
            eps=eps_rad,
            min_samples=1,
            algorithm='ball_tree',
            metric='haversine'
        ).fit(coords)
        labels[has_coords.values] = db.labels_
        offset = labels.max() + 1

    # Records without coords: cluster by site_name
    no_coords_idx = species_df.index[~has_coords]
    site_names = species_df.loc[no_coords_idx, 'site_name'].fillna('unknown')
    site_map = {s: i + offset for i, s in enumerate(site_names.unique())}
```

```
    for idx, site in site_names.items():
        labels[species_df.index.get_loc(idx)] = site_map[site]

    return labels


def cluster_species(species_df, radius_km, time_window_hours):
    """
    Apply spatial then temporal clustering to produce a cluster ID
    per record. Each unique cluster ID represents a likely single
    detection event.
    """
    species_df = species_df.copy().reset_index(drop=True)
    species_df['spatial_cluster'] = spatial_cluster(species_df, radius_km)

    time_labels = []
    for _, grp in species_df.groupby('spatial_cluster'):
        tl = build_time_clusters(grp, time_window_hours)
        time_labels.append(tl)

    species_df['time_cluster'] = pd.concat(time_labels)
    species_df['cluster_id'] = (
        species_df['spatial_cluster'].astype(str) + '_' +
        species_df['time_cluster'].astype(str)
    )
    return species_df


# ---------------------------------------------------------------------------
# Count estimation
# ---------------------------------------------------------------------------

def estimate_counts(clustered_df):
    """
    For each cluster: minimum individuals = max count in cluster.
    For each site across dates: maximum individuals = sum of daily maxima.
    Returns a summary DataFrame.
    """
    rows = []
    for cluster_id, grp in clustered_df.groupby('cluster_id'):
        min_count = grp['count_int'].max()
        observers = grp['observer_id'].nunique()
        dates = grp['date_only'].nunique()
        sites = grp['site_name'].unique().tolist()
        earliest = grp['datetime'].min()
        latest = grp['datetime'].max()
```

```
        is_flagged = (observers > 1)

        rows.append({
            'cluster_id':        cluster_id,
            'earliest_datetime': earliest,
            'latest_datetime':   latest,
            'sites':             ' | '.join(str(s) for s in sites),
            'observer_count':    observers,
            'record_count':      len(grp),
            'date_span_days':    dates,
            'min_individuals':   min_count,
            'flagged_for_review': is_flagged,
        })

    return pd.DataFrame(rows).sort_values('earliest_datetime')


def annual_summary(clustered_df, species, behaviour):
    """Roll up to a per-year minimum / maximum individual count."""
    clustered_df = clustered_df.copy()
    clustered_df['year'] = pd.to_datetime(
        clustered_df['date_only'].astype(str)
    ).dt.year

    yearly = []
    for year, yr_grp in clustered_df.groupby('year'):
        # Max individuals: sum of daily maxima per site
        # (most generous — assumes each day could be a different bird)
        max_total = (
            yr_grp.groupby(['site_name', 'date_only'])['count_int']
            .max().sum()
        )

        # Min individuals: per spatial cluster, take the max count seen
        # on any single day. For sedentary species at one site, this gives
        # 1 even if the bird was reported across many days.
        min_total = (
            yr_grp.groupby(['spatial_cluster', 'date_only'])['count_int']
            .max()                          # max count per cluster per day
            .groupby(level='spatial_cluster').max()  # highest single-day count p
            .sum()                          # sum across clusters
        )

        yearly.append({
            'year':             year,
            'species':          species,
            'behaviour':        behaviour,
```

```
            'min_individuals':  int(min_total),
            'max_individuals':  int(max_total),
            'cluster_count':    yr_grp['cluster_id'].nunique(),
            'total_records':    len(yr_grp),
        })

    return pd.DataFrame(yearly)


# ---------------------------------------------------------------------------
# Flagging ambiguous records for human review
# ---------------------------------------------------------------------------

def build_review_table(clustered_df, species, behaviour):
    """
    Return records from multi-observer clusters for human review.
    """
    multi_obs_clusters = (
        clustered_df.groupby('cluster_id')['observer_id']
        .nunique()
        .loc[lambda x: x > 1]
        .index
    )

    multi_obs = clustered_df[clustered_df['cluster_id'].isin(multi_obs_clusters)]

    if multi_obs.empty:
        return pd.DataFrame()

    review_cols = [
        'cluster_id', 'date_only', 'site_name',
        'observer_id', 'count_int', 'datetime',
        'latitude', 'longitude'
    ]
    available = [c for c in review_cols if c in multi_obs.columns]
    review = multi_obs[available].copy()
    review.insert(0, 'species', species)
    review.insert(1, 'behaviour', behaviour)
    review['verdict'] = ''  # blank column for reviewer to fill in
    return review.sort_values(['cluster_id', 'datetime'])


# ---------------------------------------------------------------------------
# Main pipeline
# ---------------------------------------------------------------------------

def run_pipeline(records_path, species_config_path, output_dir):
```

```
os.makedirs(output_dir, exist_ok=True)

print("Loading data...")
df = load_birdtrack(records_path, COLUMN_MAP)
species_config = load_species_config(species_config_path)

all_summaries = []
all_annual = []
all_review = []
all_clusters = []

target_species = species_config['species'].unique()
present_species = df['species'].unique()
not_found = set(target_species) - set(present_species)
if not_found:
    print(f"  Warning: {len(not_found)} species in config not found in record
          f"{', '.join(sorted(not_found))}")

for _, sp_row in species_config.iterrows():
    species = sp_row['species']
    behaviour = sp_row['behaviour']
    windows = BEHAVIOUR_WINDOWS[behaviour]

    sp_df = df[df['species'] == species].copy()
    if sp_df.empty:
        continue

    print(f"  Processing {species} ({behaviour}, {len(sp_df)} records)...")

    # Tier 1: same-visit deduplication
    sp_df = dedupe_same_visit(sp_df)

    # Tier 2/3: spatiotemporal clustering
    clustered = cluster_species(
        sp_df,
        radius_km=windows['radius_km'],
        time_window_hours=windows['time_window_hours']
    )
    clustered['species'] = species
    clustered['behaviour'] = behaviour

    # Cluster summary
    summary = estimate_counts(clustered)
    summary['species'] = species
    summary['behaviour'] = behaviour
    all_summaries.append(summary)
```

```
        # Annual totals
        annual = annual_summary(clustered, species, behaviour)
        all_annual.append(annual)

        # Review table
        review = build_review_table(clustered, species, behaviour)
        if not review.empty:
            all_review.append(review)

        all_clusters.append(clustered)

    # Write outputs
    if all_summaries:
        cluster_out = pd.concat(all_summaries, ignore_index=True)
        cluster_out.to_csv(
            os.path.join(output_dir, 'cluster_summary.csv'), index=False
        )
        print(f"\nCluster summary: {len(cluster_out)} clusters written.")

    if all_annual:
        annual_out = pd.concat(all_annual, ignore_index=True)
        annual_out.to_csv(
            os.path.join(output_dir, 'annual_counts.csv'), index=False
        )
        print(f"Annual counts: {len(annual_out)} species-year rows written.")

    if all_review:
        review_out = pd.concat(all_review, ignore_index=True)
        review_out.to_csv(
            os.path.join(output_dir, 'review_required.csv'), index=False
        )
        print(f"Review table: {len(review_out)} records flagged for human review.
    else:
        print("No records flagged for review.")

    print(f"\nAll outputs written to: {output_dir}")


# ---------------------------------------------------------------------------
# CLI entry point
# ---------------------------------------------------------------------------

if __name__ == '__main__':
    parser = argparse.ArgumentParser(
        description='Deduplicate BirdTrack rare species records.'
    )
    parser.add_argument('--input',   required=True,
```

```
                    help='Path to BirdTrack CSV export')
parser.add_argument('--species', required=True,
                    help='Path to species config CSV')
parser.add_argument('--output',  default='output/',
                    help='Output directory (default: output/)')
args = parser.parse_args()

run_pipeline(args.input, args.species, args.output)
```